

Review Report: Lean Formalization of Van Horn’s Uniqueness Theorem

Project: plausibility — Lean 4 (v4.33.1) + Mathlib (pinned v4.33.1) Paper: Kevin S. Van Horn, *From propositional logic to plausible reasoning: A uniqueness theorem*, IJAR 88 (2017) 309–332 Status: semantic theorem fully machine-verified; syntactic bridge infrastructure in place; one major derivation remaining Verification: lake build green · #print axioms on every theorem -> [propext, Classical.choice, Quot.sound] only · zero sorry/admit (grep + no sorryAx in axiom audit)

1. Executive summary

The paper’s core argument — *plausibility values are forced to be rational probabilities* — is now fully machine-checked at the semantic (finite-world) layer, together with the probability laws (Corollary 15), a consistency witness (Theorem 16), a counterexample showing R4 is not derivable from the rest, and a nonuniform-probability example. What remains is the syntactic bridge: deriving the semantic change-of-variables axiom from the paper’s raw propositional Requirements R1+R2 (Lemma 6’s fresh-symbol machinery). All infrastructure that derivation needs — formula lifting, fresh-symbol definitions with their model-count bijection, vocabulary splitting, and the diagram formula isolating a single truth assignment — is proved and waiting in Plausibility/VanHorn/.

The one-sentence content of the verified theorem:

R1+R2 (as semantic change-of-variables) erase representational differences, R3 erases common multiplicity, R4 preserves strict inclusion; after those eliminations the only information left is the rational ratio of favorable to possible cases.

2. What is proved, and where

Semantic layer (complete)

Paper	Lean declaration	File
Definition 5 (change of variables)	EventEquiv	Basic.lean
Semantic R1+R2 / R3 / R4	PlausibilitySystem fields equiv_invariance, irrelevant_product, strict_mono	Basic.lean
Lemma 6 / Corollary 8 (canonical reduction)	eventEquiv_canonical, score_eq_canonical	Canonical.lean
Lemma 10 (scale invariance)	upsilon2_scale, plus transport upsilon2_denom_eq	ScaleInvariant.lean
Y_1 well-defined	upsilon2_eq_of_rat_eq	RationalRepresentation.lean
Lemma 13 (strict monotone)	upsilon1_strictMono	RationalRepresentation.lean
Corollary 12 (iff-forms)	score_eq_iff_probability_eq, score_lt_iff_probability_lt	ProbabilityTheorem.lean
Theorem 14 (representation + order iso)	score_eq_upsilon1, plausibilityOrderIso : Rat01 $\simeq$ o $\uparrow$ sys.PlausibilityRange	ProbabilityTheorem.lean

Paper	Lean declaration	File
Theorem 16 (consistency)	countSystem (counting measure satisfies all fields)	ProbabilityTheorem.lean
Bridge to propositional problems	eventOf, score_eventOf	ProbabilityTheorem.lean

Syntactic layer (new, this session)

Content	Lean declaration	File
Atom lifting / support	Formula.mapLift, Formula.support, eval_mapLift	VanHorn/Substitution.lean
R2 semantic content: definitions preserve models	defFormula, defineVal, r2Equiv, card_models_def_and	VanHorn/Substitution.lean
R3 semantic content: vocabularies split as products	sumEquiv, card_models_sum	VanHorn/Substitution.lean
Lemma 6's Z_i (diagram isolating one assignment)	diagramLit, diagram, eval_diagram, models_diagram	VanHorn/Substitution.lean
Syntactic Requirements R1–R4	LogicalPlausibility structure with fields r1 r2 r3 r4	VanHorn/Requirements.lean
Corollary 15: complement & product rules	probability_complement, probability_conjunction (+ card_models_neg_and, models_and_and_assoc)	VanHorn/ProbabilityLaws.lean
Unsatisfiable-premise convention (separate)	extendedProbability, extended-Probability_eq_of_satisfiable	VanHorn/ProbabilityLaws.lean
R4 necessity counterexample	threeSystem with equiv_invariance, irrelevant_product, not_strictMono	VanHorn/Counterexamples.lean
Nonuniform example	biased_coin_heads : countSystem.score (8-of-10 event) = 4/5	VanHorn/Counterexamples.lean
Syntax + models + sanity	Formula, models, logicalProbability + 4 exhaustive checks	PropLogic/

3. Architecture

Plausibility.lean (root: imports everything)
 Plausibility/Basic.lean EventEquiv, PlausibilitySystem (PartialOrder P)
 Plausibility/Canonical.lean canonicalEvent, Corollary 8
 Plausibility/ScaleInvariant.lean Lemma 10
 Plausibility/RationalRepresentation.lean $\text{Rat01} = \{q : \mathbb{Q} // 0 \leq q \wedge q \leq 1\}$, epsilon_1 , Lemma 13
 Plausibility/ProbabilityTheorem.lean Theorem 14 + 16 + iff-forms + eventOf bridge
 Plausibility/PropLogic/Formula.lean syntax, eval

Plausibility/PropLogic/Semantics.lean models, Satisfiable, logicalProbability
 Plausibility/VanHorn/Substitution.lean mapLift, r2Equiv, sumEquiv, diagram
 Plausibility/VanHorn/Requirements.lean LogicalPlausibility (syntactic R1–R4)
 Plausibility/VanHorn/ProbabilityLaws.lean Corollary 15, extendedProbability
 Plausibility/VanHorn/Counterexamples.lean threeSystem (R4 violation), biased coin

Layering is the paper’s own:

Propositional problem $(A \mid X)$	[Requirements.lean — interface only]
↓ R1+R2, Lemma 6	← THE REMAINING GAP (infrastructure ready)
Finite event, m favorable of n worlds	[Basic, Canonical]
↓ R3	[ScaleInvariant]
Only the ratio m/n matters	[RationalRepresentation]
↓ R4	
Plausibility order $\cong \mathbb{Q} \cap [0,1]$	[ProbabilityTheorem]

4. Design decisions a reviewer should know

1. Range, not ambient codomain. `plausibilityOrderIso` targets `↑sys.PlausibilityRange` — the *achieved* values, ranging over world spaces of all finite sizes — never the ambient P (which may contain junk; e.g. $P = \mathbb{Q} \times \text{Bool}$ with unused (q, true) values).
2. `PartialOrder P`, linearity derived. The ambient order is only partial; that the achieved values form a chain is a *consequence* (via `upsilon1_strictMono`), matching the paper’s stronger claim.
3. `Rational [0,1]`, not real. `Rat01 := {q : ℚ // 0 ≤ q ∧ q ≤ 1}` (an abbrev, so instance resolution sees the subtype). No measure theory anywhere.
4. Universe polymorphism via `ULift`. `PlausibilitySystem` scores world spaces in `Type u`; canonical events live in `ULift (Fin n)` so they sit in the same universe. No fixed finite atom type is used for the range — the red-alert failure mode (“fixed α has only finitely many achieved values”) is structurally avoided.
5. Dependent-transport style. Where Lean’s dependent types block rewriting (`Fin (a+b)` vs `Fin n`), the proofs transport through explicit `EventEquives` (`finCongr`, `Equiv.ulift`) and defeq-tolerant calc chains rather than `rw`.
6. Honest layer labeling. `equiv_invariance` is *assumed* as the semantic consequence of R1+R2 (paper’s Corollary 9), not derived from the syntactic R1/R2. The README and this report state this; the `LogicalPlausibility` structure is the interface from which it must be derived.

5. The remaining gap — where to work next

Goal: from `lp : LogicalPlausibility P` (syntactic R1–R4 fields), construct `sys : PlausibilitySystem P` — i.e. prove, for every satisfiable X :

`score (event of A-worlds within X-worlds) := lp.value A X`

satisfies `equiv_invariance`, `irrelevant_product`, `strict_mono`.

The paper’s route (Lemma 6, four steps): for each satisfying assignment ρ_i of X , introduce a fresh symbol t_i defined as the diagram Z_i (we have `diagram + models_diagram`); rewrite the query as $t_1 \vee \dots \vee t_m$; define the original atoms back as disjunctions of t_i ; conclude by R1 (equivalence). Remaining Lean work, in dependency order:

1. Iterate R2 along Option: LogicalPlausibility.r2 handles *one* fresh symbol; Lemma 6 needs n at once. Either a chain of Option-lifts with support bookkeeping (Formula.support exists) or a direct $\text{Fin } n \oplus \alpha$ formulation.
2. Disjunction-of-diagrams lemma: $\text{models } (\bigvee \{v \in S\} \text{ diagram } v) = S$ (finite S). Straightforward from `models_diagram` by induction; not yet written.
3. value depends only on $(\#(A \wedge X), \#X)$ (Lemma 6 conclusion) — then `score_eq_canonical` transfers verbatim, closing the gap.
4. R3 syntactic $\rightarrow$ semantic: `lp.r3 + sumEquiv/card_models_sum` should give `irrelevant_product` for events within a premise’s models. The count bookkeeping is done; the transfer lemma isn’t.
5. R4 syntactic $\rightarrow$ semantic: `lp.r4 vs strict_mono` — subset inclusion among events corresponds to Entails of implications modulo the canonical reduction; needs (3).

Estimated effort: (2) is an evening; (1) is the genuinely fiddly part (support/freshness bookkeeping); (3) is the interesting proof; (4)–(5) are moderate given (3).

6. Where to pivot / restructure if you change course

- If Lemma 6’s iteration proves too painful: state the bridge under a *named intermediate hypothesis* (`change_of_variables : EventEquiv (models A) (models B)  $\rightarrow$  value A X = value B Y`) as an explicit field of an intermediate structure. The formalization remains fully proved and honestly labeled — the current semantic layer already takes exactly this shape.
- NNRat refactor: `Rat01` could become $\{q : \mathbb{Q} \geq 0 \text{ // } q \leq 1\}$ and `logicalProbability NNRat.divNat`. This deletes the `num  $\geq$  0/natAbs` plumbing in `RationalRepresentation.lean` (lemmas `num_natAbs_le_den`, the `Int.natAbs_of_nonneg` dances). Cosmetic-to-moderate churn over already-verified code; no new mathematics. Do it only if you’ll extend that file.
- Transfer the probability laws: Corollary 15 is currently proved for `logicalProbability` (the counting measure). A one-line corollary via `score_eq_upsilon1` would state the laws for *every* plausibility system’s calibration — high presentation value, trivial effort.
- `modelsOn` support invariance: prove $\#(\text{models over larger atom set})$ scales both counts by 2^k so ratios are invariant (use `sumEquiv` with $\beta := \text{Bool}$). Formalizes “counting over a bigger language doesn’t change the ratio”. Evening-sized.

7. Possible error points — what a reviewer should check hardest

1. `equiv_invariance` provenance (`Basic.lean`): it *assumes* the semantic consequence of R1+R2. Any claim that “R1–R4 $\Rightarrow$ probability” must route through the unproved bridge. This is the single most important caveat; it is documented, but a reviewer should verify no docstring overclaims.
2. `strict_mono` field semantics (`Basic.lean`): stated as $A \subset B \rightarrow \text{score } A < \text{score } B$. In this Mathlib, $A \subset B$ decomposes as $A \subseteq B \wedge \neg(B \subseteq A)$ (note: Lean’s `Finset ssubset` is *not* $\subseteq \wedge \neq$ here — see `upsilon1_strictMono`’s proof, which case-analyzes `hEq : B  $\subseteq$  A`). Statement is the intended one; the decomposition quirk only affected proof internals.
3. `native_decide` uses: `biased_coin_heads` (card of a concrete filter on `Fin 10`) and four sanity checks in `PropLogic/Semantics.lean`. `native_decide` trusts the compiler for decidable $\mathbb{N}/\text{Bool}$ facts — standard practice, but replaceable by `decide` if you want kernel-only evidence (slower to compile).
4. `diagram` depends on `Finset.univ.toList` ordering (`Substitution.lean`): the *definition* is order-dependent, but only `models_diagram = \{v\}` (order-independent) is ever used. If you later need syntactic identity of diagrams, revisit.
5. DecidableEq assumptions: `models` requires $[\text{DecidableEq } \alpha]$; all transfer lemmas carry it. A clas-

sical-variant API would drop it at the cost of computability; current choice keeps native_decide checks possible.

6. PlausibilityRange quantifies over Type u at the system's universe (ProbabilityTheorem.lean): a system fixed at universe u never scores spaces in Type v for $v \neq u$. The paper's single global function corresponds to using one u for everything (e.g. $u := 1$, since all finite types lift). If this matters to you, add a ULift-based "any universe" wrapper; the theorems as stated are per-universe.
7. countSystem at $P := \mathbb{Q}$ proves consistency of the *three structural fields* — i.e. of semantic R1+R2, R3, R4. It is not a LogicalPlausibility instance (syntactic R1–R4) until the bridge exists; the paper's Theorem 16 covers the syntactic version. Again a labeling subtlety, not a mathematical gap.
8. Subagent-authored files (ProbabilityLaws.lean, Counterexamples.lean, PropLogic/): independently re-audited here — recompiled (lake env lean exit 0), grep-clean of sorry/admit, and statements read line-by-line for honesty (no weakened hypotheses, no vacuous quantifier tricks). threeSystem.not_strictMono was promoted from an anonymous example to a named theorem during audit. Residual risk: low but non-zero; the two files are short and readable.

8. How to rebuild and verify

```
export PATH="$HOME/.elan/bin:$PATH"
lake build # full build (green as of this report)
lake env lean Plausibility/ProbabilityTheorem.lean # single-file type-check example
grep -R -n -E '\b(sorry|admit)\b' . --include='*.lean' --exclude-dir=.lake # must print nothing
```

Axiom audit (all results currently): #print axioms <name> -> [propext, Classical.choice, Quot.sound]
— the three standard Lean axioms; no sorryAx, no custom axioms; R1–R4 enter only as explicit structure fields.

9. Inventory at a glance

- 11 Lean files, ~1,400 lines of proof source.
- 30+ named theorems/defs across semantic and syntactic layers; every one axiom-clean.
- Two commits: d5a783f (semantic layer), 90e73ff (bridge infrastructure + laws + counterexamples).